

Análise Sintática e Gramáticas

Conceitos Estruturais, Derivações e Ambiguidade

Compiladores — Livro Base: Aho, Lam, Sethi, Ullman (ALSU) (Livro do Dragão)

Introdução à Análise Sintática

O objetivo central da análise sintática é verificar se o código fonte fornecido é **bem-formado** de acordo com as regras estruturais da linguagem.

Exemplos de Validação Estrutural

- ▶ **Cadeira Bem-Formada:** `for (i = 1; i <= n; i++) { some = some + i; mult = mult * i; }`
- ▶ **Cadeira Mal-Formada:** `for i = 1; i <= n; i++ { }` → *Ausência de parênteses delimitadores.*

O Papel do Analisador Sintático e Classificações

O analisador sintático interage de forma mútua com as demais fases do front-end do compilador, podendo executar verificações semânticas em paralelo (coleta e checagem de tipos).

Três Famílias de Métodos de Análise

- ▶ **Métodos Universais:** Algoritmos como Cocke-Younger-Kasami (CYK) e o método de Earley. Funcionam para qualquer gramática livre de contexto, porém são ineficientes para uso em compiladores reais devido à sua alta complexidade de tempo.
- ▶ **Métodos Top-Down (De Cima para Baixo):** Constrõem a árvore de derivação partindo da raiz (símbolo inicial) em direção às folhas (símbolos terminais/tokens).
- ▶ **Métodos Bottom-Up (De Baixo para Cima):** Constrõem a árvore de derivação partindo das folhas em direção à raiz da árvore.

Definição de Gramática Livre de Contexto

Uma Gramática Livre de Contexto é formalizada por uma tupla com quatro elementos: um conjunto de não-terminais, um conjunto de terminais, um símbolo inicial e regras de produção.

Exemplo Prático de Produções

Seja a gramática G definida pelo símbolo inicial E , não-terminal $\{E\}$ e terminais $\{+, *, -, (,), \text{ident}\}$, operando com as seguintes produções:

$$E \rightarrow E + E \mid E * E \mid -E \mid (E) \mid \text{ident}$$

Definição de Gramática Livre de Contexto

Definições

- ▶ **Forma Sentencial:** Se $S \xRightarrow{*} \alpha$, onde S é o símbolo inicial, dizemos que α é uma forma sentencial da gramática.
- ▶ **Sentença:** Se uma forma sentencial α não possui nenhum símbolo não-terminal (composta apenas por terminais), dizemos que ela é uma **sentença** da linguagem.

Definição de Derivações Canônicas

Durante o processo de análise sintática, quando uma forma sentencial possui múltiplos símbolos não-terminais, o compilador adota uma estratégia estrita para escolher qual variável expandir.

Derivação Mais à Esquerda ($\Rightarrow_{lm}$)

A estratégia de derivação mais à esquerda (*leftmost derivation*) determina que o símbolo não-terminal posicionado *mais à esquerda* em qualquer forma sentencial deve ser obrigatoriamente o escolhido para a próxima substituição.

- ▶ **Notação de Passos:** Representada pelo símbolo $\Rightarrow_{lm}$.
- ▶ **Uso Prático:** É a abordagem de varredura natural utilizada por analisadores sintáticos descendentes (*Top-Down*), como o analisador sintático $LL(1)$.

Definição de Derivações Canônicas

Durante o processo de análise sintática, quando uma forma sentencial possui múltiplos símbolos não-terminais, o compilador adota uma estratégia estrita para escolher qual variável expandir.

Derivação Mais à Direita ($\Rightarrow_{rm}$)

A estratégia de derivação mais à direita (*rightmost derivation*) determina que o símbolo não-terminal posicionado *mais à direita* em qualquer forma sentencial deve ser o escolhido para a próxima substituição.

- ▶ **Notação de Passos:** Representada pelo símbolo $\Rightarrow_{rm}$.
- ▶ **Uso Prático:** É a base teórica fundamental para o funcionamento de analisadores sintáticos ascendentes (*Bottom-Up*), como os analisadores sintáticos $LR(1)$ e $LALR$.

Estratégias de Derivação

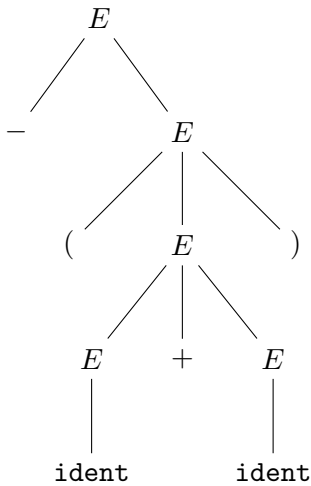
Exemplo Prático ($\Rightarrow_{lm}$) para a sentença: $-(\text{ident} + \text{ident})$

$$E \Rightarrow_{lm} -E \Rightarrow_{lm} -(E) \Rightarrow_{lm} -(E + E) \Rightarrow_{lm} -(\text{ident} + E) \Rightarrow_{lm} -(\text{ident} + \text{ident})$$

Nota: Existe um mapeamento bijetivo (um-para-um) direto entre uma Árvore de Derivação e sua respectiva derivação mais à esquerda (o mesmo valendo, de forma isolada, para a mais à direita).

Árvores de Derivação

A árvore de derivação fornece uma representação gráfica abstrata e geométrica de como uma cadeia foi decomposta através das produções da gramática.



O Problema da Ambiguidade Sintática

Definição de Ambiguidade

Uma gramática é classificada como **ambígua** se ela for capaz de produzir duas ou mais árvores de derivação distintas para uma mesma sentença da linguagem.

Considere a nossa gramática original: $E \rightarrow E + E \mid E * E \mid \text{ident}$

E a sentença: $\text{ident} + \text{ident} * \text{ident}$

O Problema da Ambiguidade Sintática

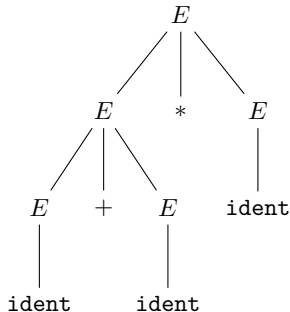
Definição de Ambiguidade

Uma gramática é classificada como **ambígua** se ela for capaz de produzir duas ou mais árvores de derivação distintas para uma mesma sentença da linguagem.

Considere a nossa gramática original: $E \rightarrow E + E \mid E * E \mid \text{ident}$

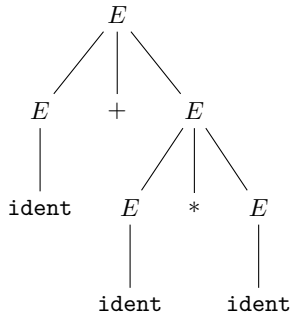
E a sentença: $\text{ident} + \text{ident} * \text{ident}$

Árvore de Derivação A



Derivações

Árvore de Derivação B



Tratamento de Ambiguidade via Reescrita

Podemos reescrever as produções introduzindo conceitos hierárquicos de precedência.

Estruturação em Camadas

Substituímos o operador genérico por variáveis que ditam a força de ligação matemática:

- ▶ **Expressão (E):** Gerencia operações de menor precedência (Adição/Subtração).
- ▶ **Termo (T):** Gerencia operações de maior precedência (Multiplicação/Divisão).
- ▶ **Fator (F):** Unidades básicas (`ident` ou expressão entre parênteses).

A Nova Gramática Não-Ambígua

Mapeamento estrito das produções baseadas na precedência matemática tradicional:

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow (E) \mid \text{ident}$$

O Problema da Ambiguidade do `else` Solto

O problema clássico do *dangling-else* ocorre em gramáticas onde a associação de uma cláusula condicional opcional gera múltiplos caminhos de derivação.

A Gramática de Comandos Ambígua (G)

Sejam os não-terminais $\{stmt, expr\}$ e os terminais minúsculos $\{if, then, else, outro, e1, e2\}$ operando sob as regras:

$$\begin{aligned} stmt &\rightarrow if\ expr\ then\ stmt \\ &\quad | if\ expr\ then\ stmt\ else\ stmt \\ &\quad | outro \\ expr &\rightarrow e1\ | e2 \end{aligned}$$

O Problema da Ambiguidade do `else` Solto

O problema clássico do *dangling-else* ocorre em gramáticas onde a associação de uma cláusula condicional opcional gera múltiplos caminhos de derivação.

A Gramática de Comandos Ambígua (G)

Sejam os não-terminais $\{stmt, expr\}$ e os terminais minúsculos $\{if, then, else, outro, e1, e2\}$ operando sob as regras:

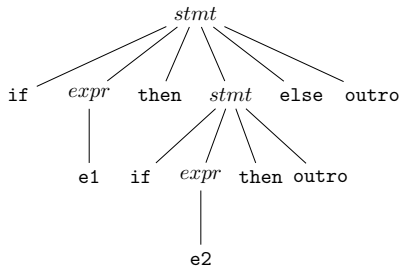
$$\begin{aligned} stmt &\rightarrow if\ expr\ then\ stmt \\ &\quad | if\ expr\ then\ stmt\ else\ stmt \\ &\quad | outro \\ expr &\rightarrow e1\ | e2 \end{aligned}$$

Prova. Via sentença: `if e1 then if e2 then outro else outro`

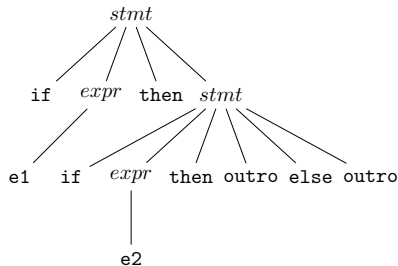
Árvores de Derivação Concorrentes (*Dangling-Else*)

A ambiguidade da gramática condicional produz duas estruturas de árvores sintáticas distintas para a mesma cadeia de entrada.

1ª Interpretação (else externo)



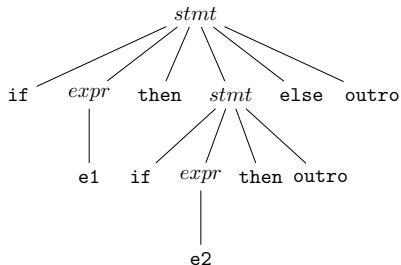
2ª Interpretação (else interno)



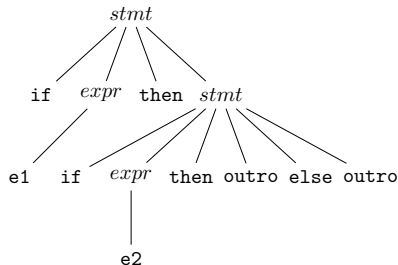
Árvores de Derivação Concorrentes (*Dangling-Else*)

A ambiguidade da gramática condicional produz duas estruturas de árvores sintáticas distintas para a mesma cadeia de entrada.

1ª Interpretação (else externo)



2ª Interpretação (else interno)



Como resolver? Emparelhar cada cláusula else com o token then mais próximo e mais interno que ainda não tenha sido emparelhado (forçando a validade estrita da **2ª Interpretação**).